Sri Lanka Institute of Information Technology

IT2130 – Operating Systems and System Administration
Year 2, Semester 2- 2026

Practical Answer Submission

Practical Sheet No: 04

Student ID	IT24101803
Student Name	Harikaran T.
Campus/ Center name	Malabe
Specialization	ISE
Batch No	01.01

Activity 1

a. What is the output you can see in the terminal?

```
  GNU nano 6.2                                  lab4.c
#include<stdio.h>
#include<unistd.h>
int main()
{
   printf("Process ID is: %d\n", getpid());
   return 0;
}
```

```
Terminal
$ nano lab4.c
$ gcc -olab4 lab4.c
$ ./lab4
Process ID is: 3863
$
```

b. What is the system call used in the above program?

- getpid()

c. Briefly describe the purpose of the system call you mentioned in a)?

- It returns the process ID (PID) of the currently running process.

d. Remove the include statement, ”#include¡unistd.h¿” from the program and check the output. What can you see in the output? Describe the reason for your observation.

```
ubuntu@ubuntu:~$ nano lab4.c
ubuntu@ubuntu:~$ gcc -olab4 lab4.c
lab4.c: In function ‘main’:
lab4.c:4:34: warning: implicit declaration of function ‘getpid’ [-Wimplicit-func
tion-declaration]
    4 |     printf("Process ID is: %d\n", getpid());
      |                                   ^~~~~~
ubuntu@ubuntu:~$
```

- getpid() is declared inside <unistd.h>. Without it, the compiler doesn't know the function prototype.

Activity 2

```
GNU nano 7.2
#include <stdio.h>
#include <unistd.h>

int main() {
    printf("Process ID: %d\n", getpid());
    printf("Parent Process ID: %d\n", getppid());
    return 0;
}
```

```
Terminal
$ nano act2.c
$ gcc -oact2 act2.c
$ ./act2
Current Process ID: 6683
Parent Process ID: 6654
$
```

Activity 3

1)

```
GNU nano 7.2                act3.c
#include <stdio.h>
#include <unistd.h>

int main() {
    printf("Process ID: %d\n", getpid());
    printf("Parent Process ID: %d\n", getppid());

    sleep(30);
    return 0;
}
```

```
ubuntu@ubuntu: ~
ubuntu@ubuntu:~$ nano act3.c
ubuntu@ubuntu:~$ gcc -oact3 act3.c
ubuntu@ubuntu:~$ ./act3
Process ID: 7598
Parent Process ID: 7563
ubuntu@ubuntu:~$
```

Activity 4

```
ubuntu@ubuntu:~$ nano act4.c
ubuntu@ubuntu:~$ gcc -oact4 act4.c
ubuntu@ubuntu:~$ ./act4
Program is running...
ubuntu@ubuntu:~$
```

- This will only print “program is running…” Because exit(0) immediately terminates the process. Any code after exit() will never execute.

Activity 5

```
ubuntu@ubuntu:~$ nano act5.c
ubuntu@ubuntu:~$ gcc -oact5 act5.c
ubuntu@ubuntu:~$ ./act5
I am parent
Hello world
Hello world
ubuntu@ubuntu:~$
```

- Because fork() creates a child process , Both parent and child execute printf(). So "Hello world" prints two times

Activity 6

Modify the program in activity 5, to do the below.

1. Invoke the fork ( ) system call to create a child process.

2. Print the return value of the fork ( ).

```
GNU nano 7.2
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <stdlib.h>

int main() {
    pid_t pid = fork(); // Creates a child process

    // Step 2: Print the return value of fork()
    printf("Return value of fork: %d\n", pid);

}
```

```
ubuntu@ubuntu:~$ nano act6.c
ubuntu@ubuntu:~$ gcc -oact6 act6.c
ubuntu@ubuntu:~$ ./act6
Return value of fork: 9145
ubuntu@ubuntu:~$ Return value of fork: 0
```

3. Separate the parent and the child processes and print their process IDs as shown in figure below.

```c
// Step 3: Separate the parent and child process
    if (pid == 0) {
        // This is the child process
        printf("Child Process\n");
        printf("Child PID: %d\n", getpid());
        printf("Parent PID (Child): %d\n", getppid());
    } else if (pid > 0) {
        // This is the parent process
        printf("Parent Process\n");
        printf("Parent PID: %d\n", getpid());
        printf("Child PID (Parent): %d\n", pid);
    } else {

        perror("fork failed");
        exit(1);
    }

}
```

```
ubuntu@ubuntu:~$ nano act6.c
ubuntu@ubuntu:~$ gcc -oact6 act6.c
ubuntu@ubuntu:~$ ./act6
Return value of fork: 0
Return value of fork: 9335
Parent Process
Parent PID: 9334
Child PID (Parent): 9335
Child Process
Child PID: 9335
Parent PID (Child): 2009
ubuntu@ubuntu:~$
```

4. Add the sleep ( ) with an appropriate time in seconds and check the process hierarchy.

```c
  // Step 4: Sleep to check the process hierarchy
    sleep(20);
    return 0;
}
```

```
ubuntu@ubuntu:~$ nano act6.c
ubuntu@ubuntu:~$ gcc -oact6 act6.c
ubuntu@ubuntu:~$ ./act6
Return value of fork: 0
Child Process
Child PID: 9412
Parent PID (Child): 9411
Return value of fork: 9412
Parent Process
Parent PID: 9411
Child PID (Parent): 9412
```

Activity 7

```
ubuntu@ubuntu:~
ubuntu@ubuntu:~$ nano act7.c
ubuntu@ubuntu:~$ gcc -oact7 act7.c
ubuntu@ubuntu:~$ ./act7
Here comes the date
Desktop  Documents  Downloads  Music  Pictures  Public  Templates  Videos  act3  act3.c  act4  act4.c  act5  act5.c  act6  act6.c  act7  act7.c  lab4  lab4.c  snap
ubuntu@ubuntu:~$
```

- The execl() system call replaces the current process image with a new program. After execl() is successfully executed, the original program does not continue execution. That is why the statement after execl() is not printed. The output shown is the result of the ls command executed by execl().

Activity 8

```
ubuntu@ubuntu:~$ nano act8.c
ubuntu@ubuntu:~$ gcc -oact8 act8.c
ubuntu@ubuntu:~$ ./act8
Here comes the date.
Sat Feb 21 16:30:48 UTC 2026
That was the date.
ubuntu@ubuntu:~$
```

- The system() function executes a shell command by creating a child process internally. The parent process waits until the command completes. Therefore, the date command is executed once, and the program prints the messages before and after the system() call in sequential order.

Modify the above program by adding fork () prior to system () function. Observe the output and explain the output.

```
  GNU nano 7.2
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int main() {

   fork();

   printf("Here comes the date.\n");
   system("date");
   printf("That was the date.\n");
   return 0;
}
```

```
ubuntu@ubuntu:~$ nano act8.c
ubuntu@ubuntu:~$ gcc -oact8 act8.c
ubuntu@ubuntu:~$ ./act8
Here comes the date.
Here comes the date.
Sat Feb 21 16:36:44 UTC 2026
That was the date.
ubuntu@ubuntu:~$ Sat Feb 21 16:36:44 UTC 2026
That was the date.
```

- When fork() is added before system(), a child process is created. Both parent and child processes execute the remaining code independently. Therefore, the printf statements and the system("date") command are executed twice. Since both processes run concurrently, the output order may vary each time the program runs.